

TẬP ĐOÀN CÔNG NGHIỆP – VIỆN THÔNG QUÂN ĐỘI



BÁO CÁO BÀI LAB

CSAPP : DATALAB

Họ và Tên : Lê Chí Hiếu
Email : ximoncute147@gmail.com
Đơn vị : Viettel High Tech
Lĩnh vực : Embedded Linux

HÀ NỘI, 2026

Mục lục

1	PHẦN 1: HƯỚNG DẪN CHẠY LỆNH NHANH (QUICK START)	3
1.1	Bước 1: Cài đặt thư viện hỗ trợ biên dịch 32-bit (Chỉ chạy 1 lần duy nhất)	3
1.2	Bước 2: Di chuyển vào thư mục bài làm	3
1.3	Bước 3: Dọn dẹp tệp tin cũ & Biên dịch lại	3
1.4	Bước 4: Chạy thử nghiệm tất cả các hàm để kiểm tra kết quả	3
1.5	Bước 5: Chạy tự động chấm điểm toàn bộ (Cả điểm chính xác và số toán tử)	3
2	PHẦN 2: CÁC KHÁI NIỆM CƠ BẢN CẦN BIẾT	4
3	PHẦN 3: GIẢI THÍCH CHI TIẾT TỪNG HÀM TRONG BITS.C	5
3.1	Hàm 1: bitXor(int x, int y)	5
3.2	Hàm 2: tmin(void)	6
3.3	Hàm 3: isTmax(int x)	7
3.4	Hàm 4: allOddBits(int x)	8
3.5	Hàm 5: negate(int x)	9
3.6	Hàm 6: isAsciiDigit(int x)	10
3.7	Hàm 7: conditional(int x, int y, int z)	11
3.8	Hàm 8: isLessOrEqual(int x, int y)	12
3.9	Hàm 9: logicalNeg(int x)	13
3.10	Hàm 10: howManyBits(int x)	14
3.11	Hàm 11: floatScale2(unsigned uf)	16
3.12	Hàm 12: floatFloat2Int(unsigned uf)	18
3.13	Hàm 13: floatPower2(int x)	20

GIỚI THIỆU

Bài thực hành CS:APP Data Lab (`bits.c`) là một trong những bài tập kinh điển giúp người học làm quen và hiểu sâu sắc về cách biểu diễn dữ liệu nhị phân của các kiểu số nguyên có dấu (số bù 2) và số thực dấu phẩy động (chuẩn IEEE 754). Thông qua việc giải quyết các bài toán thao tác bit với những ràng buộc khắt khe về số lượng và loại toán tử được phép sử dụng, chúng ta có thể rèn luyện tư duy logic toán học tối ưu, hiểu rõ bản chất cơ chế tính toán vật lý của vi xử lý.

1 PHẦN 1: HƯỚNG DẪN CHẠY LỆNH NHANH (QUICK START)

Bạn chỉ cần copy đúng các lệnh dưới đây vào terminal WSL (hoặc Ubuntu) để cài đặt, biên dịch và chạy thử.

1.1 Bước 1: Cài đặt thư viện hỗ trợ biên dịch 32-bit (Chỉ chạy 1 lần duy nhất)

Bài lab này yêu cầu biên dịch ở kiến trúc 32-bit (-m32). Bạn hãy copy dòng lệnh dưới đây và chạy trong WSL:

```
1 sudo apt update && sudo apt install -y build-essential gcc-multilib g++-multilib perl
```

1.2 Bước 2: Di chuyển vào thư mục bài làm

```
1 cd /mnt/c/Users/Admin/Desktop/Viettel_high_tech/01-data-lab/datalab-handout
```

1.3 Bước 3: Dọn dẹp tệp tin cũ & Biên dịch lại

Mỗi lần bạn sửa đổi file `bits.c`, bạn bắt buộc phải chạy lệnh này để biên dịch lại chương trình thử nghiệm:

```
1 make clean && make btest
```

1.4 Bước 4: Chạy thử nghiệm tất cả các hàm để kiểm tra kết quả

Chạy lệnh dưới đây để kiểm tra xem bạn được bao nhiêu điểm. Chúng tôi đã tăng thời gian chờ lên 30 giây để tránh lỗi quá hạn (timeout):

```
1 ./btest -T 30
```

1.5 Bước 5: Chạy tự động chấm điểm toàn bộ (Cả điểm chính xác và số toán tử)

Chạy lệnh này để kiểm tra xem bạn có vi phạm quy định nào không (như dùng quá số lượng toán tử, dùng lệnh cấm) và hiển thị bảng điểm cuối cùng:

```
1 ./driver.pl
```

Kết quả đúng tuyệt đối sẽ hiển thị: Score = 62/62 [36/36 Corr + 26/26 Perf]

2 PHẦN 2: CÁC KHÁI NIỆM CƠ BẢN CẦN BIẾT

Nếu bạn chưa hiểu tại sao lại viết code như vậy, hãy đọc phần này:

1. **Phép AND (&):** Giữ lại các bit 1 ở những vị trí mà cả hai số đều là 1.
 - Ví dụ: $1100 \& 1010 = 1000$.
2. **Phép OR (|):** Bật bit thành 1 nếu ít nhất một trong hai số ở vị trí đó là 1.
 - Ví dụ: $1100 | 1010 = 1110$.
3. **Phép XOR (^):** Trả về 1 nếu hai bit khác nhau, trả về 0 nếu hai bit giống nhau.
 - Ví dụ: $1100 \wedge 1010 = 0110$.
4. **Phép NOT (~):** Đảo ngược tất cả các bit (0 thành 1, 1 thành 0).
 - Ví dụ: $\sim 1100 = 0011$.
5. **Phép dịch trái (« n):** Đẩy các bit sang trái n vị trí, thêm n bit 0 vào bên phải (Tương đương nhân với 2^n).
6. **Phép dịch phải số học (» n):** Đẩy các bit sang phải n vị trí. Với số có dấu, các ô trống bên trái sẽ được điền bằng bit dấu (bit 1 nếu là số âm, bit 0 nếu là số dương).
 - Ví dụ số âm: $11110000 \gg 2 = 11111100$.
 - Ví dụ số dương: $00001100 \gg 2 = 00000011$.
7. **Phép phủ định logic (!):** Trả về 1 nếu số đó bằng 0, trả về 0 nếu số đó khác 0.
 - Ví dụ: $!0 = 1, !5 = 0, !-100 = 0$.
8. **Số bù 2 (Số nguyên có dấu):**
 - Bit ngoài cùng bên trái (bit 31) là bit dấu. Nếu nó bằng 1 $\rightarrow$ số âm, nếu bằng 0 $\rightarrow$ số dương hoặc 0.
 - Giá trị đối của x (tức $-x$) luôn luôn bằng $\sim x + 1$ (đảo tất cả các bit rồi cộng 1).
 - Số nhỏ nhất biểu diễn được (Tmin): $0x80000000$ (chỉ có bit dấu là 1).
 - Số lớn nhất biểu diễn được (Tmax): $0x7FFFFFFF$ (bit dấu bằng 0, tất cả các bit còn lại là 1).

3 PHẦN 3: GIẢI THÍCH CHI TIẾT TỪNG HÀM TRONG BITS.C

3.1 Hàm 1: bitXor(int x, int y)

- **Yêu cầu đề bài:** Hãy tính phép toán $x \hat{~} y$ (XOR) nhưng bạn chỉ được dùng hai toán tử là $\sim$ (NOT) và $\&$ (AND).
- **Giới hạn toán tử cho phép:** Tối đa 14. Số toán tử chúng ta dùng: 7.
- **Ý tưởng logic đơn giản:**

- Định nghĩa phép XOR là: Trả về các bit mà x và y khác nhau.
- Tức là: (x là 1 VÀ y là 0) HOẶC (x là 0 VÀ y là 1).
- Biểu thức logic: $(x \& \sim y) \mid (\sim x \& y)$.
- Nhưng đề bài cấm dùng phép OR ($\mid$). Chúng ta phải chuyển phép OR thành phép AND bằng cách dùng định lý De Morgan:

$$A \vee B = \neg(\neg A \wedge \neg B)$$

Thay $A = (x \wedge \neg y)$ và $B = (\neg x \wedge y)$, ta được biểu thức chỉ có $\sim$ và $\&$:

$$\sim(\sim(x \& \sim y) \& \sim(\sim x \& y))$$

- Có một cách viết khác gọn hơn và chạy nhanh hơn:
 - * $\sim(x \& y)$: Lọc bỏ những vị trí mà cả hai cùng bằng 1.
 - * $\sim(\sim x \& \sim y)$: Lọc bỏ những vị trí mà cả hai cùng bằng 0 (vì đảo của 0 là 1, nếu cả hai là 0 thì $\sim x \& \sim y$ bằng 1, đảo lại được 0).
 - * Kết hợp hai cái bằng phép AND: $\sim(x \& y) \& \sim(\sim x \& \sim y)$.

- **Giải thích code chi tiết:**

```
1 int bitXor(int x, int y) {  
2     //  $\sim(x \& y)$ : Nhưng bit KHÔNG ĐỒNG THỜI bằng 1 ở cả  $x$  và  $y$   
3     //  $\sim(\sim x \& \sim y)$ : Nhưng bit KHÔNG ĐỒNG THỜI bằng 0 ở cả  $x$  và  $y$   
4     // AND hai giá trị này lại ta sẽ giữ được những bit mà  $x$  và  
5     //  $y$  khác nhau.  
6     return  $\sim(x \& y) \& \sim(\sim x \& \sim y)$ ;  
}
```

- **Ví dụ minh họa:** Giả sử $x = 4$ (nhị phân 0100), $y = 5$ (nhị phân 0101).

- $x \& y = 0100 \& 0101 = 0100$.
- $\sim(x \& y) = \sim 0100 = 1011$.
- $\sim x = 1011, \sim y = 1010$.
- $\sim x \& \sim y = 1011 \& 1010 = 1010$.
- $\sim(\sim x \& \sim y) = \sim 1010 = 0101$.
- Kết quả: $1011 \& 0101 = 0001$ (bằng 1). Đúng bằng $4 \hat{~} 5 = 1$.

3.2 Hàm 2: tmin(void)

- **Yêu cầu đề bài:** Trả về số nguyên âm nhỏ nhất có thể biểu diễn bằng hệ bù 2 32-bit.
- **Giới hạn toán tử cho phép:** Tối đa 4. Số toán tử chúng ta dùng: 1.
- **Ý tưởng logic đơn giản:**
 - Số nguyên âm nhỏ nhất (Tmin) trong hệ 32-bit là số có dạng nhị phân: 1000 0000 0000 0000 0000 0000 0000 (chữ số 1 ở vị trí bit thứ 31, và 31 vị trí còn lại là số 0).
 - Hệ cơ số 16 của số này là 0x80000000.
 - Ta có thể tạo ra số này bằng cách lấy số 1 dịch sang trái 31 bit.
- **Giải thích code chi tiết:**

```
1 int tmin(void) {  
2     // Lay so 1 (nhị phân ...0001) dịch trái 31 lần để đưa bit  
   1 lên vị trí bit đầu cao nhất.  
3     return 1 << 31;  
4 }
```

3.3 Hàm 3: isTmax(int x)

- **Yêu cầu đề bài:** Trả về 1 nếu x đúng bằng số nguyên dương lớn nhất ($T_{\max} = 0x7FFFFFFF$), ngược lại trả về 0. Bạn không được phép sử dụng toán tử dịch bit (« hay »).
- **Giới hạn toán tử cho phép:** Tối đa 10. Số toán tử chúng ta dùng: 6.
- **Ý tưởng logic đơn giản:**
 - Số $T_{\max}$ là $0x7FFFFFFF$ (nhị phân: bit dấu bằng 0, toàn bộ 31 bit còn lại bằng 1).
 - Nếu ta cộng thêm 1 vào $T_{\max}$, nó sẽ tràn số thành $T_{\min}$ ($0x80000000$, chỉ có bit dấu là 1, còn lại là 0).
 - Lúc này, nếu ta lấy số đảo bit của $T_{\max}$ (tức là $\sim T_{\max}$), ta cũng thu được $0x80000000$.
 - Như vậy, với số $T_{\max}$, ta luôn có thuộc tính đặc biệt: $(x + 1)$ bằng đúng $\sim x$.
 - Phép XOR $\wedge$ giữa hai số bằng nhau sẽ cho kết quả bằng 0: $(x + 1) \wedge \sim x == 0$.
 - Trường hợp ngoại lệ: Số -1 (nhị phân toàn bit 1: $0xFFFFFFFF$) cũng có tính chất này! Vì $-1 + 1 = 0$ và đảo bit của -1 ($\sim(-1)$) cũng bằng 0. Nên $(0 \wedge 0) == 0$.
 - Vì vậy, ta phải loại trừ số -1 . Ta biết rằng nếu $x = -1$ thì $x + 1 = 0$. Ta dùng phép phủ định logic $!(x + 1)$ để kiểm tra. Nếu $x + 1$ bằng 0, biểu thức này trả về 1, ngược lại trả về 0.
 - Ta kết hợp điều kiện loại trừ bằng phép OR $|$. Chỉ khi cả hai điều kiện cùng bằng 0 thì ta mới xác định đó đúng là $T_{\max}$.

- **Giải thích code chi tiết:**

```
1 int isTmax(int x) {
2     int x_plus_1 = x + 1; // Tao gia tri x + 1
3     // (x_plus_1 ^ ~x): Kiem tra xem x + 1 co bang ~x khong.
4     // Neu bang nhau, ket qua se la 0.
5     // !x_plus_1: Loai tru so -1. Neu x = -1 thi x+1 = 0, phu
6     // dinh lai duoc 1.
7     // Phép | ket hop: Neu x dung la Tmax, ca hai ve deu la 0,
8     // 0 | 0 = 0.
9     // Phép ! ngoai cung bien ket qua 0 thanh 1 (dung), cac gia
10    tri khac thanh 0 (sai).
11    return !((x_plus_1 ^ ~x) | !x_plus_1);
12 }
```

3.4 Hàm 4: allOddBits(int x)

- **Yêu cầu đề bài:** Kiểm tra xem toàn bộ các bit ở vị trí lẻ (bit 1, 3, 5, ..., 31) của số x có đồng thời bằng 1 hay không. Nếu có thì trả về 1, nếu thiếu dù chỉ một bit thì trả về 0.
- **Giới hạn toán tử cho phép:** Tối đa 12. Số toán tử chúng ta dùng: 7.
- **Ý tưởng logic đơn giản:**
 - Ta cần tạo một số mặt nạ (mask) mà toàn bộ các bit lẻ bằng 1, bit chẵn bằng 0. Số đó là 0xAAAAAAAA (nhị phân: 1010 1010...).
 - Đề bài cấm viết hằng số lớn hơn 255. Do đó ta không thể viết trực tiếp 0xAAAAAAAA. Ta phải ghép nó từ số nhỏ 0xAA (nhập hệ 10 là 170).
 - Ta dịch bit 0xAA sang trái 8 bit rồi OR lại để được 0xAAAA. Dịch tiếp 0xAAAA sang trái 16 bit rồi OR lại để được 0xAAAAAAAA.
 - Để kiểm tra x có đủ các bit lẻ hay không, ta lấy $x \& \text{mask}$. Phép toán này sẽ giữ lại các bit lẻ của x và xóa hết các bit chẵn về 0.
 - Nếu các bit lẻ của x đều bằng 1, thì kết quả của $x \& \text{mask}$ phải giống hệt như mask .
 - Ta so sánh hai số này bằng phép XOR $\wedge$. Nếu giống nhau hoàn toàn, phép XOR sẽ trả về 0. Ta phủ định ! kết quả này để trả về 1.
- **Giải thích code chi tiết:**

```
1 int allOddBits(int x) {
2     int mask = 0xAA | (0xAA << 8); // Tao ra 0xAAAA (16 bit le
    bang 1)
3     mask = mask | (mask << 16);    // Nhan doi len thanh 0
    xAAAAAAAA (32 bit le bang 1)
4
5     // (x & mask): Loc lay cac bit o vi tri le trong x.
6     // ... ^ mask: So sanh phan loc duoc voi mask goc. Neu khop
    hoan toan, ket qua bang 0.
7     // !(...): Bien ket qua 0 thanh 1 de tra ve ket qua Dung.
8     return !((x & mask) ^ mask);
9 }
```

3.5 Hàm 5: negate(int x)

- **Yêu cầu đề bài:** Trả về giá trị số đối $-x$. Ví dụ đầu vào là 5 thì trả về -5, đầu vào là -10 thì trả về 10.
- **Giới hạn toán tử cho phép:** Tối đa 5. Số toán tử chúng ta dùng: 2.
- **Ý tưởng logic đơn giản:**
 - Trong máy tính sử dụng biểu diễn số nguyên bù 2, số đối của số x luôn bằng phép đảo bit cộng 1.
 - Công thức toán học toán tử bit: $-x = \sim x + 1$.
- **Giải thích code chi tiết:**

```
1 int negate(int x) {  
2     // Đảo ngược toàn bộ bit của x (NOT x) rồi cộng thêm 1.  
3     return ~x + 1;  
4 }
```

3.6 Hàm 6: isAsciiDigit(int x)

- **Yêu cầu đề bài:** Kiểm tra xem giá trị của x có nằm trong khoảng mã ASCII từ $0x30$ (số 48 trong hệ thập phân, ký tự '0') đến $0x39$ (số 57, ký tự '9') hay không. Nếu nằm trong khoảng này trả về 1, ngược lại trả về 0.
- **Giới hạn toán tử cho phép:** Tối đa 15. Số toán tử chúng ta dùng: 13.
- **Ý tưởng logic đơn giản:**
 - Một số x nằm trong khoảng $[0x30, 0x39]$ khi và chỉ khi:
 - * $x \geq 0x30 \rightarrow$ phép tính $x - 0x30 \geq 0$.
 - * $x \leq 0x39 \rightarrow$ phép tính $0x39 - x \geq 0$ (hoặc tương đương $x - 0x3A < 0$).
 - Trong máy tính, phép trừ được biểu diễn bằng phép cộng số đối: $A - B = A + \sim B + 1$.
 - Ta kiểm tra kết quả phép tính có âm hay không bằng cách dịch phải 31 bit để lấy bit dấu. Nếu kết quả âm, bit dấu sẽ là 1 (hoặc -1 nếu dùng dịch phải số học). Nếu kết quả dương hoặc bằng 0, bit dấu là 0.
 - Trường hợp tràn số: Nếu x là một số âm rất lớn hoặc dương rất lớn, phép trừ có thể bị tràn làm đảo lộn bit dấu. Do đó, ta phải kiểm tra thêm: bit dấu của x phải bằng 0 (tức x không phải số âm). Ta lấy $\text{sign} = x \gg 31$. Nếu $x \geq 0$ thì sign bằng 0.
- **Giải thích code chi tiết:**

```
1 int isAsciiDigit(int x) {
2     int sign = x >> 31;                // Lay bit dau cua x.
3     Neu x am, sign se bang -1. Neu x >= 0, sign = 0.
4     int upper = (x + ~0x3A + 1) >> 31; // Tinh hieu x - 0x3A.
5     Neu x < 0x3A (tuc <= 0x39), hieu nay am, bit dau bang -1 (
6     nhi phan toan bit 1).
7     int lower = (x + ~0x30 + 1) >> 31; // Tinh hieu x - 0x30.
8     Neu x >= 0x30, hieu nay khong am, bit dau bang 0.
9
10    // !sign: Tra ve 1 neu x >= 0 (loai tru so am tranh tran so
11    ).
12    // upper: Tra ve -1 (toan bit 1) neu x <= 0x39.
13    // !lower: lower bang -1 neu x < 0x30. Phu dinh !lower bang
14    1 neu x >= 0x30.
15    // AND ket hop ca ba dieu kien lai.
16    return !sign & upper & !lower;
17 }
```

3.7 Hàm 7: conditional(int x, int y, int z)

- **Yêu cầu đề bài:** Tái lập lại toán tử ba ngôi $x ? y : z$. Nếu x khác 0 thì trả về y , nếu x bằng 0 thì trả về z .
- **Giới hạn toán tử cho phép:** Tối đa 16. Số toán tử chúng ta dùng: 7.
- **Ý tưởng logic đơn giản:**
 - Ta muốn tạo ra một chiếc "mặt nạ lọc"(mask) dựa trên x .
 - Nếu $x \neq 0$, mặt nạ phải bằng $0xFFFFFFFF$ (toàn bộ bit 1). Khi đó ta dùng $y \& \text{mask}$ để lấy toàn bộ bit của y .
 - Nếu $x == 0$, mặt nạ phải bằng $0x00000000$ (toàn bộ bit 0). Khi đó ta dùng $z \& \sim \text{mask}$ (tức $z \& 0xFFFFFFFF$) để lấy toàn bộ bit của z .
 - Cách tạo mặt nạ:
 - * Dùng phép phủ định logic $!x$:
 - Nếu $x \neq 0 \rightarrow !x = 0$.
 - Nếu $x == 0 \rightarrow !x = 1$.
 - * Lấy kết quả đó cộng với ~ 0 (tức số -1, có biểu diễn nhị phân là toàn bộ bit 1):
 - Nếu $x \neq 0$: $0 + (-1) = -1$ (mặt nạ nhị phân toàn 1).
 - Nếu $x == 0$: $1 + (-1) = 0$ (mặt nạ nhị phân toàn 0).

- **Giải thích code chi tiết:**

```
1 int conditional(int x, int y, int z) {
2     int mask = !x + ~0; // Tạo mặt nạ: bằng 0xFFFFFFFF nếu x
   khác 0; bằng 0x00000000 nếu x bằng 0.
3
4     // (y & mask): Giữ lại y nếu mask toàn 1, bằng 0 nếu mask
   toàn 0.
5     // (z & ~mask): Giữ lại z nếu mask toàn 0 (khi đó ~mask
   toàn 1), bằng 0 nếu mask toàn 1.
6     // Phép | ghép hai phần lại để trả về giá trị mong muốn.
7     return (y & mask) | (z & ~mask);
8 }
```

3.8 Hàm 8: isLessOrEqual(int x, int y)

- **Yêu cầu đề bài:** Trả về 1 nếu $x \leq y$, ngược lại trả về 0.
- **Giới hạn toán tử cho phép:** Tối đa 24. Số toán tử chúng ta dùng: 16.
- **Ý tưởng logic đơn giản:**
 - Thông thường để so sánh $x \leq y$, ta chỉ việc lấy hiệu $y - x$ và kiểm tra xem kết quả có lớn hơn hoặc bằng 0 hay không.
 - Tuy nhiên, trong kiểu số nguyên có giới hạn, phép trừ có thể gây ra hiện tượng tràn số (overflow).
 - * Ví dụ: x là số âm cực lớn (Tmin), và y là số dương lớn. Hiệu $y - x$ sẽ là phép cộng hai số dương lớn, vượt quá giới hạn dương và quay về số âm. Khi đó phép so sánh dấu sẽ bị sai.
 - Để giải quyết triệt để, ta chia làm 2 trường hợp:
 1. Khác dấu: x và y khác dấu nhau.
 - * Nếu x âm ($sx = 1$) and y dương ($sy = 0$), chắc chắn $x < y$ là đúng $\rightarrow$ trả về 1.
 - * Nếu x dương ($sx = 0$) and y âm ($sy = 1$), chắc chắn $x > y \rightarrow$ trả về 0.
 2. Cùng dấu: x and y cùng dương hoặc cùng âm.
 - * Khi cùng dấu, phép trừ $y - x$ chắc chắn không bao giờ bị tràn số.
 - * Ta tính hiệu $diff = y - x$ (tức $y + \sim x + 1$).
 - * Lấy bit dấu của hiệu: $diff_sign = (diff \gg 31) \& 1$.
 - * Nếu hiệu ≥ 0 (tức $diff_sign == 0$) thì $x \leq y$ đúng $\rightarrow$ trả về 1.

- **Giải thích code chi tiết:**

```
1 int isLessOrEqual(int x, int y) {
2     int sx = (x >> 31) & 1; // Lay bit dau cua x (1 neu am, 0
   neu khong am)
3     int sy = (y >> 31) & 1; // Lay bit dau cua y (1 neu am, 0
   neu khong am)
4
5     // Tinh hieu y - x va lay bit dau cua ket qua.
6     int diff_sign = ((y + ~x + 1) >> 31) & 1;
7
8     int same_sign = !(sx ^ sy); // Tra ve 1 neu x va y cung dau
   , 0 neu khac dau.
9
10    // (same_sign & !diff_sign): Neu cung dau, ket qua dung khi
   hieu y-x khong am (diff_sign == 0).
11    // (!same_sign & sx): Neu khac dau, ket qua dung khi x la
   so am (sx == 1).
12    return (same_sign & !diff_sign) | (!same_sign & sx);
13 }
```

3.9 Hàm 9: logicalNeg(int x)

- **Yêu cầu đề bài:** Tự thiết kế toán tử logic phủ định !x (trả về 1 nếu x bằng 0, trả về 0 nếu x khác 0) mà không được dùng chính toán tử !.
- **Giới hạn toán tử cho phép:** Tối đa 12. Số toán tử chúng ta dùng: 5.
- **Ý tưởng logic đơn giản:**
 - Ta cần tận dụng một đặc điểm duy nhất của số 0:
 - * Số 0 và số đối của nó (là -0 hay $\sim x + 1$) đều có bit dấu bằng 0.
 - * Với mọi số khác 0 (kể cả số âm và số dương), một trong hai số x hoặc -x sẽ luôn là số âm (tức là bit thứ 31 bằng 1).
 - Ví dụ:
 - * Nếu $x = 5$ (dương), số đối -5 có bit dấu bằng 1.
 - * Nếu $x = -5$ (âm), nó có bit dấu bằng 1.
 - * Nếu $x = \text{Tmin}(-2147483648)$, số đối của nó cũng là Tmin, có bit dấu bằng 1.
 - Do đó, ta lấy phép OR giữa x và số đối -x (tức $\sim x + 1$): $x \mid (\sim x + 1)$.
 - * Nếu $x \neq 0$: Biểu thức này chắc chắn có bit thứ 31 bằng 1.
 - * Nếu $x == 0$: Biểu thức này bằng $0 \mid 0 = 0$ (bit thứ 31 bằng 0).
 - Ta dịch phải số học 31 bit để đưa bit dấu này phủ toàn bộ số:
 - * Nếu $x \neq 0 \rightarrow$ dịch phải được $0xFFFFFFFF$ (tức số -1).
 - * Nếu $x == 0 \rightarrow$ dịch phải được $0x00000000$ (tức số 0).
 - Cộng thêm 1 vào kết quả để đưa về dạng logic chuẩn 0 hoặc 1:
 - * Nếu khác 0: $-1 + 1 = 0$.
 - * Nếu bằng 0: $0 + 1 = 1$.
- **Giải thích code chi tiết:**

```
1 int logicalNeg(int x) {
2     // (~x + 1) là số đối của x.
3     // (x | số đối): Lay bit dấu. Luôn có bit 31 bằng 1 nếu x
   khác 0.
4     // >> 31: Dịch phải số học để tạo số toàn bit 1 (-1) hoặc
   toàn bit 0 (0).
5     // + 1: Chuyển đổi -1 thành 0, và chuyển đổi 0 thành 1.
6     return ((x | (~x + 1)) >> 31) + 1;
7 }
```

3.10 Hàm 10: `howManyBits(int x)`

- **Yêu cầu đề bài:** Tính số bit tối thiểu cần thiết để biểu diễn giá trị của số `x` dưới dạng số nguyên bù 2 (bao gồm cả bit dấu).
- **Giới hạn toán tử cho phép:** Tối đa 90. Số toán tử chúng ta dùng: 32.
- **Ý tưởng logic đơn giản:**
 - Với số dương, ví dụ số 5 (nhị phân 0101), ta cần ít nhất 4 bit để biểu diễn. Số bit tối thiểu phụ thuộc vào vị trí của bit 1 cao nhất.
 - Với số âm, ví dụ số -5 (nhị phân 1011), số bit tối thiểu cần dùng cũng tương tự như số dương đảo bit của nó (tức là số 4, nhị phân 0100 $\rightarrow$ cần 4 bit).
 - Do đó, bước 1: Nếu `x` âm, ta đảo toàn bộ bit của nó. Nếu `x` dương, ta giữ nguyên. Ta làm việc này bằng phép XOR với bit dấu: `x = x ^ (x > 31)`.
 - Bước 2: Ta đi tìm vị trí bit 1 cao nhất bằng kỹ thuật Tìm kiếm nhị phân trên 32 bit:
 - * Kiểm tra xem nửa trên (16 bit cao) có chứa bit 1 nào không:
`b16 = !(x > 16) < 4;`
Nếu nửa trên có bit 1, ta dịch phải `x` đi 16 bit để tiếp tục tìm kiếm trong 16 bit đó, đồng thời ghi nhận đã tìm thấy ở khoảng 16 bit (`b16 = 16`). Ngược lại `b16 = 0`.
 - * Tương tự kiểm tra tiếp trong phần còn lại cho khoảng 8 bit (`b8`), 4 bit (`b4`), 2 bit (`b2`), và 1 bit (`b1`).
 - * Cuối cùng cộng tất cả các khoảng dịch lại, cộng thêm 1 bit cho vị trí bit 0 và 1 bit dấu.
- **Giải thích code chi tiết:**

```

1 int howManyBits(int x) {
2     int sign, b16, b8, b4, b2, b1, b0;
3     sign = x >> 31; // Tao mat na dau: 0xFFFFFFFF neu x am, 0
                       x00000000 neu x duong.
4     x = x ^ sign;   // Neu x am, dao bit cua x. Neu x duong,
                       giu nguyen x.
5
6     // Tim kiem nhi phan bit 1 cao nhat:
7     b16 = (!(x >> 16)) << 4; // Neu x > 0xFFFF, b16 = 16,
                       nguoc lai bang 0.
8     x = x >> b16;           // Dich phai x de thu hep pham vi
                       tim kiem.
9
10    b8 = (!(x >> 8)) << 3;   // Neu x con lai > 0xFF, b8 = 8,
                       nguoc lai bang 0.
11    x = x >> b8;
12
13    b4 = (!(x >> 4)) << 2;   // Neu x con lai > 0xF, b4 = 4.
14    x = x >> b4;
15
16    b2 = (!(x >> 2)) << 1;   // Neu x con lai > 0x3, b2 = 2.
17    x = x >> b2;
18
19    b1 = (!(x >> 1));        // Neu x con lai > 0x1, b1 = 1.
20    x = x >> b1;
21
22    b0 = x;                  // Bit cuoi cung con lai.
23
24    // Tong so bit can dung = tong cac buoc dich + 1 bit co so
                       + 1 bit dau.
25    return b16 + b8 + b4 + b2 + b1 + b0 + 1;
26 }

```

3.11 Hàm 11: floatScale2(unsigned uf)

- **Yêu cầu đề bài:** Đầu vào là biểu diễn bit của một số thực đơn đơn độ chính xác chuẩn IEEE 754. Hãy nhân số đó với 2 và trả về biểu diễn bit của kết quả.
- **Quy tắc:** Đây là phần số thực nên bạn được phép dùng cấu trúc rẽ nhánh `if`, vòng lặp, và các hằng số bất kỳ.
- **Giới hạn toán tử cho phép:** Tối đa 30. Số toán tử chúng ta dùng: 13.
- **Ý tưởng logic đơn giản (IEEE 754):**
 - Một số thực 32-bit gồm 3 phần:
 - * 1 bit dấu ở vị trí 31 (**sign**).
 - * 8 bit mũ ở vị trí 30-23 (**exp**).
 - * 23 bit phân số ở vị trí 22-0 (**frac**).
 - Khi nhân số thực này với 2, ta xét các trường hợp:
 1. Trường hợp đặc biệt ($\text{exp} = 0xFF$): Số này là NaN (Not a Number) hoặc Vô cùng (∞). Theo chuẩn, vô cùng hay NaN nhân 2 vẫn giữ nguyên giá trị $\rightarrow$ trả về chính nó.
 2. Trường hợp số phi chuẩn ($\text{exp} = 0$): Số này cực kỳ nhỏ, gần bằng 0. Để nhân đôi, ta chỉ cần dịch trái phần phân số đi 1 bit: **frac** $\ll$ 1. Khi dịch trái, nếu bit cao nhất của phân số vượt ngưỡng, nó sẽ tự tràn vào phần mũ (làm phần mũ lên 1), biến số đó thành số chuẩn hóa một cách hoàn toàn chính xác.
 3. Trường hợp số chuẩn hóa ($0 < \text{exp} < 0xFF$): Ta chỉ cần tăng phần mũ lên 1 đơn vị (**exp++**). Nếu sau khi tăng mà phần mũ đạt giá trị tràn `0xFF`, số này trở thành vô cùng, ta đặt phần phân số **frac** = 0.
- **Giải thích code chi tiết:**

```

1 unsigned floatScale2(unsigned uf) {
2     unsigned sign = uf & 0x80000000; // Trich xuất bit dấu
3     unsigned exp = (uf >> 23) & 0xFF; // Trich xuất 8 bit mũ
4     unsigned frac = uf & 0x007FFFFF; // Trich xuất 23 bit phần
      so
5
6     if (exp == 0xFF) return uf;          // Neu là NaN hoặc
      Infinity, tra về nguyên bản.
7
8     if (exp == 0) {
9         // Neu là số phi chuẩn (exp = 0), dịch trái uf 1 bit (giữ
      nguyên bit dấu).
10        return sign | (uf << 1);
11    }
12
13    exp++; // Số chuẩn hóa bình thường: tăng phần mũ lên 1 (
      tương đương nhân 2).
14    if (exp == 0xFF) {
15        frac = 0; // Neu vượt ngưỡng mũ tối đa, biến thành số Vô
      cùng (frac = 0).
16    }
17
18    // Gộp các thành phần lại và tra về.
19    return sign | (exp << 23) | frac;
20 }

```

3.12 Hàm 12: floatFloat2Int(unsigned uf)

- **Yêu cầu đề bài:** Thực hiện chuyển đổi ép kiểu từ số thực (float) sang số nguyên (int). Nếu giá trị float vượt quá giới hạn biểu diễn của số nguyên hoặc là NaN, trả về 0x80000000.
- **Giới hạn toán tử cho phép:** Tối đa 30. Số toán tử chúng ta dùng: 14.
- **Ý tưởng logic đơn giản:**
 - Công thức tính giá trị số thực chuẩn hóa: $V = (-1)^{\text{sign}} \times (1.\text{frac}) \times 2^E$, trong đó số mũ thực tế là $E = \text{exp} - 127$.
 - Ta xét các trường hợp của E :
 - * Nếu $E < 0$ ($\text{exp} < 127$): Trị tuyệt đối của số thực này nhỏ hơn 1.0 (ví dụ 0.5, -0.9). Khi ép sang kiểu nguyên int, kết quả luôn bị làm tròn về 0.
 - * Nếu $E \geq 31$ ($\text{exp} \geq 158$): Vượt quá khả năng biểu diễn của số nguyên 32-bit có dấu (tối đa là $2^{31} - 1$). Trả về giá trị lỗi 0x80000000.
 - * Nếu $0 \leq E < 31$: Ta khôi phục phần định trị đầy đủ bằng cách thêm số 1 ẩn vào trước phần phân số: $M = 1.\text{frac}$ (tương ứng mã code là `frac | 0x00800000`).
 - Phần định trị M này đang có 23 bit sau dấu phẩy.
 - Nếu $E \geq 23$: Ta cần dịch trái thêm để bù vào phần nguyên: `val = M << (E - 23)`.
 - Nếu $E < 23$: Ta dịch phải để loại bỏ các bit thập phân dư thừa: `val = M >> (23 - E)`.
 - Cuối cùng, nếu số thực ban đầu mang dấu âm (`sign` bằng 1), ta đảo dấu giá trị số nguyên thu được (`val = -val`).
- **Giải thích code chi tiết:**

```

1 int floatFloat2Int(unsigned uf) {
2     unsigned sign = uf & 0x80000000;
3     unsigned exp = (uf >> 23) & 0xFF;
4     unsigned frac = uf & 0x007FFFFFFF;
5     int E = exp - 127; // Tinh so mu thuc te
6     int M;
7     int val;
8
9     if (exp < 127) return 0; // Tri tuyet doi nho hon 1 -> lam
    tron ve 0.
10    if (exp >= 158) return 0x80000000u; // Vuot qua gioi han so
    nguyen 32-bit -> loi.
11
12    M = frac | 0x00800000; // Khoi phuc bit an 1 thanh 1.frac (
    tong cong 24 bit dinh tri)
13    if (E >= 23) {
14        val = M << (E - 23); // So mu lon hon so bit phan so ->
    dich trai de tang gia tri.
15    } else {
16        val = M >> (23 - E); // So mu nho hon 23 -> dich phai de
    luoc bo phan thap phan.
17    }
18
19    if (sign) {
20        val = -val; // Neu so thuc am, doi dau so nguyen ket qua.
21    }
22    return val;
23 }

```

3.13 Hàm 13: floatPower2(int x)

- **Yêu cầu đề bài:** Trả về biểu diễn nhị phân (dưới dạng float) của biểu thức lũy thừa 2.0^x với số nguyên x bất kỳ.
- **Giới hạn toán tử cho phép:** Tối đa 30. Số toán tử chúng ta dùng: 9.
- **Ý tưởng logic đơn giản:**
 - Ta cần tính biểu diễn bit của số thực 2.0^x :
 - * Giá trị lớn nhất mà số thực chuẩn hóa biểu diễn được là khoảng 2^{127} . Nếu $x > 127$, kết quả bị tràn số $\rightarrow$ trả về Vô cùng dương +INF (0x7F800000).
 - * Giá trị nhỏ nhất có thể biểu diễn được dưới dạng số phi chuẩn là 2^{-149} . Nếu $x < -149$, số quá nhỏ $\rightarrow$ làm tròn về 0.
 - * Khoảng số chuẩn hóa $[-126, 127]$: Giá trị phần mũ lưu trữ trong bộ nhớ sẽ là $\text{exp} = x + 127$. Phần phân số **frac** bằng 0. Ta trả về $(x + 127) \ll 23$.
 - * Khoảng số phi chuẩn $[-149, -127]$: Phần mũ **exp** bằng 0. Giá trị được biểu diễn duy nhất qua phần phân số **frac**. Bit 1 sẽ được dịch chuyển tới vị trí thứ $x + 149$. Ta trả về $1 \ll (x + 149)$.
- **Giải thích code chi tiết:**

```
1 unsigned floatPower2(int x) {
2     if (x > 127) return 0x7F800000; // Vuot gioi han tren ->
    Tra ve duong vo cung.
3     if (x < -149) return 0;         // Vuot gioi han duoi ->
    Tra ve 0.
4
5     if (x >= -126) {
6         // So thuc chuan hoa: phan mu luu gia tri (x + 127), phan
        phan so bang 0.
7         return (x + 127) << 23;
8     }
9
10    // So thuc phi chuan: phan mu bang 0, chi co 1 bit 1 nam o
    phan phan so.
11    return 1 << (x + 149);
12 }
```

LỜI CẢM ƠN & KẾT LUẬN

Báo cáo xin gửi lời cảm ơn chân thành đến tập thể các anh chị hướng dẫn tại Viettel High Tech đã tạo điều kiện, hỗ trợ và định hướng kỹ thuật trong suốt quá trình hoàn thành bài lab. Những kiến thức thu nhận được từ bài lab Data Lab này sẽ là nền tảng vững chắc cho quá trình học tập và nghiên cứu chuyên sâu về lĩnh vực Embedded Linux sau này.